Anisa / Iran Linux House

طرح درس

MLOps Fundamentals On-Prem

۱۴ جلسه‌ی ۳ ساعته · ۴۰ ساعت

مدرس

سعیدرضا سالاری

Saeed Reza Salari

نسخه‌ی ۲٫۰ · تاریخ: ۱۰-۲۰۲۶

فهرست مطالب

Table of Contents

۱ شناسنامه‌ی دوره 3
۲ مخاطبان و پیش‌نیازها 3
۳ اهداف یادگیری دوره 5
۴ روش تدریس 5
۵ ابزارها و پشته‌ی فنی 7
۶ نمای کلی دوره 8
۷ برنامه‌ی جلسه‌به‌جلسه 10
جلسه ۰ پیش‌کار غیرحضوری 10
جلسه ۱ شروع دوره و مبانی MLOps 11
جلسه ۲ Git برای ML و کیفیت کد 12
جلسه ۳ مبانی Docker و ذخیره‌ساز شیء On-Prem 13
جلسه ۴ نسخه‌بندی داده با DVC 14
جلسه ۵ DVC Pipeline و بازتولیدپذیری 15
جلسه ۶ MLflow Tracking و اتصال به پروژه 16
جلسه ۷ Model Registry و Promotion 17
جلسه ۸ سرویس Inference با FastAPI 18
جلسه ۹ Containerization و امنیت image 19
جلسه ۱۰ Deploy روی VM و Infrastructure as Code 20
جلسه ۱۱ Kubernetes Deploy 21
جلسه ۱۲ Ingress و Helm 22
جلسه ۱۳ CI آگاه از ML، Monitoring و Drift 23
جلسه ۱۴ جمع‌بندی و Demo پروژه‌ی پایانی (Mini Production) 24
۸ ارزیابی 25
۹ خروجی نهایی دوره 26
۱۰ هم‌ترازی با roadmap.sh/mlops 26
۱۱ وضعیت تولید محتوا 27

شماره‌ی صفحه‌ها (متن ثابت) با نسخه‌ی PDF همین سند هماهنگ است. برای رفتن به هر بخش روی عنوان کلیک کنید.

۱ شناسنامه‌ی دوره

Course Identity

عنوان دوره	MLOps Fundamentals On-Prem (مبانی MLOps در محیط On-Prem)
مدت دوره	حدود ۴۰ ساعت آموزش حضوری (۱۴ جلسه × ۲ ساعت و ۵۰ دقیقه خالص)، به‌اضافه‌ی جلسه‌ی ۰ غیرحضوری حدود ۱ ساعت و ۴۵ دقیقه
تعداد و طول جلسات	۱۴ جلسه‌ی ۳ ساعته (۲ ساعت و ۵۰ دقیقه خالص + ۱۰ دقیقه استراحت)؛ جلسه‌ی ۰ پیش‌کار غیرحضوری
سطح	مقدماتی تا متوسط؛ هدف: نقش Junior MLOps Engineer، یا مهندس ML/DevOps که وارد MLOps می‌شود
زبان مطالب	دوزبانه: متن فارسی؛ اصطلاحات، نام ابزارها و دستورها انگلیسی
مدل اجرا	لپ‌تاپ دانشجو + بسته‌ی آفلاین (imageها، wheelhouse پایتون، باینری‌ها)؛ کلاس به اینترنت وابسته نیست. Windows از طریق WSL2
پروژه‌ی محوری	Churn Mini Production: پیش‌بینی ریزش مشتری (جدولی، scikit-learn) با داده‌ی مصنوعی تکرارپذیر؛ یک Repo که هر جلسه بخشی از آن را جلو می‌برد
ارزیابی	کاملاً عملی: ۲۰٪ checkpointهای جلسه‌ها، ۳۰٪ تمرین‌های Lab، ۵۰٪ پروژه‌ی پایانی
مجری	Iran Linux House / Anisa
مدرس	سعیدرضا سالاری

۲ مخاطبان و پیش‌نیازها

Audience & Prerequisites

مخاطبان

مهندسان ML، مهندسان DevOps/SRE و برنامه‌نویسانی که می‌خواهند وارد MLOps شوند، و کسانی که هدفشان نقش Junior MLOps Engineer است.

خروجی دوره: دانشجو می‌تواند یک سرویس ML قابل‌تکرار، نسخه‌دار، ردیابی‌شده، کانتینری و پایش‌شده را به‌تنهایی روی زیرساخت On-Prem بسازد؛ تقریباً سطح بلوغ ۱ (pipeline خودکار + CI، با promotion دستی).

پیش‌نیازهای دانشی

- Python در حد توابع، محیط مجازی و pip
- آشنایی کلی با ML در حد ساخت یک مدل ساده با scikit-learn (مدل در این دوره «هدف» نیست، فقط «موضوع سرویس‌دهی» است)

- کار پایه با terminal (در جلسه‌ی ۰ مرور می‌شود)
- آشنایی با Git (clone و commit) مفید است ولی الزامی نیست

هدف دوره نیست: پلتفرم‌های ML ابری، big data و streaming، سرویس‌دهی GPU/DL، edge، Feature Store و GitOps (بخشی در دوره‌ی ۲ آمده است).

پیش‌نیاز سخت‌افزاری و نرم‌افزاری

هر دانشجو با لپ‌تاپ خودش کار می‌کند. حداقل و مقدار پیشنهادی:

مورد	حداقل	پیشنهادی
CPU	۴ هسته، x86-64 یا Apple Silicon، با فعال بودن VT-x/AMD-V	۶ تا ۸ هسته
RAM	۸ گیگابایت (با «حالت رم کم»)	۱۶ گیگابایت
فضای آزاد دیسک	۲۵ گیگابایت	۴۰ گیگابایت SSD
سیستم‌عامل	Ubuntu 22.04+، macOS 13+، یا Windows 10 22H2 / 11 با WSL2	Ubuntu 24.04 یا macOS 14+
دسترسی	دسترسی admin/sudo برای نصب Docker و Multipass	—

- Windows: اجرای همه‌ی دستورها داخل WSL2 با Ubuntu 24.04 و Docker Engine نصب‌شده در WSL. macOS: Colima به‌جای Docker Desktop. Ubuntu: Docker Engine CE نسخه‌ی ۲۷ به بالا.
- اوج مصرف RAM حدود ۷ تا ۹ گیگابایت و در جلسه‌های ۱۲ و ۱۳ است. لپ‌تاپ‌های ۸ گیگابایتی با اجرای فقط سرویس‌های جلسه‌ی جاری کار می‌کنند.
- لپ‌تاپ‌های سازمانی اغلب دسترسی admin یا مجازی‌سازی ندارند؛ اسکریپت preflight.sh باید دو هفته قبل از شروع روی همه‌ی لپ‌تاپ‌ها اجرا شود.
- نسخه‌ی همه‌ی ابزارها در فایل `lab-kit/versions.env` قفل شده است (تاریخ قفل: ۲۰۲۶-۱۰-۰۸).

۳ اهداف یادگیری دوره

Learning Outcomes

در پایان دوره دانشجو می‌تواند:

۱ مسیر یک پروژه‌ی ML را از Notebook تا Production توضیح دهد و نقش‌ها، ۷ جزء MLOps و سطوح بلوغ ۰ تا ۲ را بشناسد.

۲ یک Repo استاندارد با محیط قفل‌شده (`uv.lock`) و تنظیمات بیرون از کد بسازد و یک baseline را به‌صورت تکرارپذیر اجرا کند.

۳ با Git و Gitea (branch، Pull Request، tag نسخه) کار کند و با ruff، pytest و pre-commit کیفیت کد ML را خودکار و secretها را مسدود کند.

۴ سرویس‌ها را با Docker و Compose اجرا کند و ذخیره‌ساز S3-compatible داخلی (SeaweedFS) را راه بیندازد و با AWS CLI بیازماید.

۵ داده و pipeline آموزش را با DVC نسخه‌بندی و تکرارپذیر کند و به هر نسخه‌ی داده برگردد.

۶ runها را با MLflow ثبت و مقایسه کند، مدل را در Registry ثبت کند و با alias و دروازه‌ی ارزیابی promote کند.

۷ مدل را با FastAPI سرویس‌دهی کند، image امن (multi-stage، non-root، اسکن Trivy) بسازد و در registry محلی push کند.

۸ سرویس را روی VM با Ansible و روی Kubernetes (kind) با Helm و Ingress مستقر کند و rollback را عملاً انجام دهد.

۹ CI روی Gitea Actions، monitoring با Prometheus/Grafana و drift report با Evidently را برپا کند و پروژه را با Runbook و Demo تحویل دهد.

۴ روش تدریس

Teaching Method

- Project-First، یک Repo واحد: از جلسه‌ی ۱ پروژه‌ی Churn Mini Production شروع می‌شود و هر جلسه یک بخش مشخص (Deliverable) به همان Repo اضافه می‌کند. مدل «موضوع» است، نه «هدف».
- هر جلسه: حداکثر ۲ مفهوم اصلی با Demo، سپس یک Lab هدایت‌شده‌ی ۷۵ دقیقه‌ای با پنج گام زمان‌دار، و پایان جلسه با Definition of Done.
- عقب‌ماندگی: Repo مرجع مدرس در پایان هر جلسه tag دارد (`s01-end` تا `s14-end`)؛ دانشجویی که Lab را تمام نکرده از همان tag ادامه می‌دهد. قاعده‌ی کلاس: اول از بغل‌دستی بپرسید، بعد دست بالا؛ بیش از ۱۰ دقیقه گیر کردید، از tag ادامه دهید.
- Checkpoint خودکار: هر جلسه یک اسکریپت verify دارد (مثلاً `verify.sh s01`)؛ خروجی آن و commit hash، شاهد نمره‌ی checkpoint است.

- تمرین شکست عمدی (Failure Injection): دانشجو عمداً سیستم را خراب می‌کند، مثلاً کلید S3 یا پورت اشتباه، probe حذف‌شده، tag نادرست image، تغییر داده بدون `dvc repro`؛ و پیام‌ها را مستند می‌کند. عیب‌یابی بخش اصلی مهارت شغلی است.
- کلاس بدون اینترنت: همه‌ی imageها، بسته‌های پایتون، باینری‌ها و داده از بسته‌ی آفلاین می‌آیند؛ جلسه‌ی ۰ برای نصب و بررسی preflight است تا زمان جلسه‌ی ۱ حفظ شود.

قالب ثابت هر جلسه

قالب هر جلسه‌ی ۳ ساعته ثابت است (۲ ساعت و ۵۰ دقیقه خالص + ۱۰ دقیقه استراحت):

زمان	مدت	بلاک	محتوا
۰۰:۱۵–۰۰:۰۰	۱۵ دقیقه	**شروع**	مرور جلسه‌ی قبل، بررسی تکلیف، اهداف جلسه
۰۱:۲۰–۰۰:۱۵	۶۵ دقیقه	**مفاهیم + Demo**	بلاک ۱: حداکثر ۲ مفهوم اصلی با Demo زنده
۰۱:۳۰–۰۱:۲۰	۱۰ دقیقه	**استراحت**	—
۰۲:۴۵–۰۱:۳۰	۷۵ دقیقه	**Lab**	بلاک ۲: Lab هدایت‌شده روی Repo دوره، پنج گام زمان‌دار
۰۳:۰۰–۰۲:۴۵	۱۵ دقیقه	**جمع‌بندی**	Definition of Done، آزمون کوتاه (۴ سؤال)، تکلیف و پیش‌نمای جلسه‌ی بعد

در جلسه‌های ساخته‌شده، مجموع زمان پنج گام Lab کمی کمتر از ۷۵ دقیقه است؛ باقی‌مانده، زمان انعطاف برای رفع اشکال است.

۵ ابزارها و پشته‌ی فنی

Tools & Technical Stack

همه‌ی ابزارها متن‌باز و self-hosted هستند و نسخه‌ها قفل شده‌اند. ستون «جلسه» جلسه‌ی اولین استفاده را نشان می‌دهد.

حوزه	ابزار	نسخه‌ی قفل‌شده	جلسه
محیط Python	uv، Python 3.12	uv 0.12.15	۱
نسخه‌بندی کد و Git server	Git، Gitea	Gitea 1.27.3	۲
کیفیت کد	ruff، pytest، pre-commit	ruff 0.16.7، pytest 9.1.1	۲
کانتینر	Docker، Docker Compose	Docker Engine ≥ 27	۳، ۹
ذخیره‌ساز شیء (S3)	SeaweedFS، AWS CLI	SeaweedFS 4.45	۳
نسخه‌بندی داده و pipeline	DVC	dvc 3.67.1	۴، ۵
Registry و Tracking	MLflow (alias و tag)	mlflow 3.16.0	۶، ۷
سرویس‌دهی مدل	FastAPI، pydantic، httpx	fastapi 0.141.1	۸
امنیت image و registry	Trivy، Syft (اختیاری)، registry محلی	Trivy 0.74.0	۹
استقرار روی VM و IaC	Ansible، Multipass، Nginx، systemd	ansible-core 2.21.4	۱۰
Kubernetes	kind، kubectl	kind v0.33.0، K8s v1.36.4	۱۱
Ingress و بسته‌بندی	Traefik، Helm	Traefik chart 41.5.0، Helm v4.2.4	۱۲
CI	Gitea Actions (act_runner)	act_runner 0.6.1	۱۳
Monitoring	Prometheus، Grafana	kube-prometheus-stack 91.3.0	۱۳
Drift	Evidently	evidently 0.7.23	۱۳

۶ نمای کلی دوره

Course Overview

جلسه	عنوان	مفاهیم اصلی	خروجی Lab
۰	**پیش‌کار غیرحضوری**	مبانی Linux/Bash: shell، pipe، env، مجوزها، ssh، curl	نصب از بسته‌ی آفلاین، preflight.sh تا READY
۱	**شروع دوره و مبانی MLOps**	چرا MLOps، ۷ جزء، سطوح بلوغ، استاندارد Repo	Repo اولیه + baseline قابل اجرا
۲	**Git برای ML و کیفیت کد**	branch، PR، tag، ruff، pytest، pre-commit	Repo روی Gitea با ۴ آزمون، hook و tag `v0.1.0`
۳	**مبانی Docker و ذخیره‌ساز S3 داخلی**	image، container، volume، Compose، S3	SeaweedFS با Compose و دو bucket (dvc، mlflow)
۴	**نسخه‌بندی داده با DVC**	pointer، cache، remote، زمان‌سفر، lineage	remote فعال و دو نسخه‌ی داده با tag
۵	**DVC Pipeline و بازتولیدپذیری**	`dvc.yaml`، DAG، `dvc repro`، `dvc exp`، orchestration	pipeline تکرارپذیر با خروجی متریک
۶	**MLflow Tracking**	run، param، metric، artifact، معماری server	MLflow UI + runهای ثبت‌شده با artifact روی S3
۷	**Model Registry و Promotion**	alias، tag، signature، دروازه‌ی ارزیابی	مدل با alias + اسکریپت promotion
۸	**سرویس Inference با FastAPI**	REST، pydantic، health، logging، `/metrics`	endpoint فعال و آزمون‌شده
۹	**Containerization و امنیت image**	multi-stage، non-root، Trivy، SBOM، registry	image امن‌تر و نسخه‌دار در registry محلی
۱۰	**Deploy روی VM و Infrastructure as Code**	systemd، Nginx، Ansible، idempotency	سرویس روی VM + playbook + Runbook
۱۱	**Kubernetes Deploy**	Deployment، Service، ConfigMap، probe، limit	سرویس running روی kind
۱۲	**Ingress و Helm**	Traefik، Gateway API، chart، values	سرویس با Helm chart + Ingress
۱۳	**CI آگاه از ML، Monitoring و Drift**	Gitea Actions، RED، Prometheus/Grafana، Evidently	CI عملیاتی + dashboard + drift report
۱۴	**جمع‌بندی و Demo پروژه‌ی پایانی**	چک‌لیست Production readiness، Demo، roadmap	Repo نهایی + Runbook + Demo

تصویر تجمعی پروژه: Repo در هر جلسه چه چیزی اضافه می‌کند

یک Repo واحد (churn-mlops) از جلسه‌ی ۱ تا ۱۴ رشد می‌کند؛ هر ردیف فقط چیزی است که همان جلسه به Repo اضافه می‌کند.

پایه‌ی پروژه	
۱	ساختار Repo (`src`، `scripts`، `configs`، `tests`)، `params.yaml`، `Makefile`، README، baseline train
۲	remote روی Gitea، آزمون‌ها، ruff، hookهای pre-commit، tag `v0.1.0`
۳	فایل Compose سرویس‌ها، `env.example`، bucketهای dvc و mlflow
داده و pipeline	
۴	ردیابی داده با DVC (pointer در Git)، remote روی S3، tagهای `data-v1` و `data-v2`
۵	`dvc.yaml` با مراحل preprocess، train و evaluate
مدل	
۶	MLflow server با Compose و ثبت runهای pipeline
۷	مدل ثبت‌شده با alias و اسکریپت promotion با دروازه‌ی ارزیابی
۸	API با FastAPI و آزمون‌های آن
بسته‌بندی و استقرار	
۹	Dockerfile امن و image نسخه‌دار در registry محلی
۱۰	playbook Ansible برای VM و Runbook
۱۱	manifestهای K8s (Deployment، Service، ConfigMap، probe، resources)
۱۲	Ingress با Traefik و Helm chart با `values-dev` و `values-prod`
عملیات و تحویل	
۱۳	workflowهای Gitea Actions (CI و retrain زمان‌بندی‌شده)، dashboard Grafana، drift report
۱۴	Repo نهایی قابل اجرا از clone تمیز، Runbook و Demo

۷ برنامه‌ی جلسه‌به‌جلسه

Session-by-Session Plan

هر جلسه با همان قالب ثابت نوشته شده است. جلسه‌های ۱ تا ۴ طبق محتوای ساخته‌شده (اسلاید، یادداشت مدرس و Lab اجراشده) و زمان واقعی گام‌های Lab آمده‌اند؛ جلسه‌های ۵ تا ۱۴ و جلسه‌ی ۰ از نقشه‌ی دوره نوشته شده‌اند.

جلسه ۰ — پیش‌کار غیرحضوری

Pre-work (async) · Session 0 غیرحضوری · حدود ۱ ساعت و ۴۵ دقیقه

اهداف جلسه

دانشجو در پایان جلسه‌ی ۰:

مبانی Linux/Bash لازم برای همه‌ی Labها را مرور کند

بسته‌ی آفلاین را نصب کند

با اجرای preflight.sh وضعیت READY بگیرد تا جلسه‌ی ۱ صرف نصب نشود

بلاک ۱ — مفاهیم

مبانی Linux/Bash: shell، pipe، متغیر محیطی، مجوزها، ssh، curl

آزمون کوتاه خودسنجی

بلاک ۲ — Lab

نصب از بسته‌ی آفلاین

اجرای preflight.sh تا وضعیت READY

خروجی / Definition of Done

خروجی: لپ‌تاپ آماده برای جلسه‌ی ۱

preflight.sh وضعیت READY می‌دهد

آزمون کوتاه انجام شده است

تکلیف

—

ابزارها: Bash، ssh، curl، preflight.sh، بسته‌ی آفلاین

پیش‌نیاز از جلسه‌ی قبل: نصب runtime کانتینر (Docker Engine، Colima یا WSL2) و Multipass پیش از جلسه‌ی ۰ توسط دانشجو یا IT

جلسه ۱ — شروع دوره و مبانی MLOps

Session 1 · Kickoff & MLOps Foundations

اهداف جلسه

در پایان این جلسه می‌توانید:

MLOps را تعریف کند و تفاوت آن با DevOps و Data Science و جایگاه نقش MLOps Engineer را توضیح دهد

۷ جزء اصلی MLOps (طبق roadmap.sh) و ابزار متناظر هرکدام در دوره را بشناسد

وضعیت یک تیم را در سطوح بلوغ ۰ تا ۲ جای دهد و هدف دوره (سطح ۱) را بداند

Repo استاندارد بسازد و یک baseline قابل تکرار را با یک دستور اجرا کند

بلاک ۱ — مفاهیم و Demo (۶۵ دقیقه)

چرا MLOps: فاصله‌ی Notebook تا Production و خرابی بی‌صدای مدل (drift)

نقش‌ها (DS، ML Engineer، MLOps)، چرخه‌ی عمر مدل، اجزای MLOps و سطوح بلوغ ۰/۱/۲

On-Prem، محدودیت‌های Release و rollback، فعالیت خودارزیابی با roadmap.sh/mlops

استاندارد Repo: `src/ scripts/ configs/ tests/`، تنظیمات بیرون از کد، محیط قفل‌شده (`uv.lock`)، seed

معرفی پروژه‌ی Churn و روش ارزیابی

بلاک ۲ — Lab (۷۵ دقیقه)

بررسی محیط: preflight.sh باید READY باشد (۱۰ دقیقه)

ساخت Repo و skeleton پروژه با uv (`git init`، `uv init`) (۱۵ دقیقه)

تولید داده‌ی مصنوعی و پیکربندی (`make_dataset.py`، `params.yaml`) (۱۰ دقیقه)

آموزش baseline و ساخت `metrics.json` (۲۰ دقیقه)

`.gitignore`، Makefile (data، train)، README و اولین commit (۲۰ دقیقه)

خروجی / Definition of Done

خروجی: Repo اولیه + baseline قابل اجرا (ROC-AUC حدود ۰٫۸۰)

preflight.sh خروجی READY دارد

`make data` و `make train` بدون خطا اجرا می‌شوند

مدل و `metrics.json` ساخته شده‌اند و در Git نیستند

README بخش «How to run» دارد

حداقل یک commit روی شاخه‌ی main وجود دارد

تکلیف

انتقال آستانه‌ی تصمیم (threshold) از کد به `params.yaml`

کامل کردن README (هدف، پیش‌نیاز، اجرا، ساختار، متریک)

تمام کردن پیش‌مطالعه‌ی Linux/Bash (جلسه‌ی ۰) و بالا آوردن lab kit از Gitea

ابزارها: uv، Python 3.12، Makefile، scikit-learn، Git

پیش‌نیاز از جلسه‌ی قبل: لپ‌تاپ آماده با preflight.sh در وضعیت READY (جلسه‌ی ۰)

جلسه ۲ — Git برای ML و کیفیت کد

Session 2 · Git for ML + Code Quality

اهداف جلسه

در پایان این جلسه می‌توانید:

تعیین کند چه چیزی در Git می‌رود، با branch و Conventional Commits کار کند و Release را tag بزند

قالب‌بندی و lint را با ruff اجرا و تست‌های واحد معنادار برای کد ML بنویسد

با hookهای pre-commit محلی، فایل env و secret hard-code را قبل از commit مسدود کند

Repo را روی Gitea بگذارد، Pull Request باز و merge کند و tag `v0.1.0` بزند

بلاک ۱ — مفاهیم و Demo (۶۵ دقیقه)

چه چیزی در Git می‌رود و چه چیزی نه؛ مسیر working dir، staging، repo، remote

branching ساده (feature/release/hotfix)، Conventional Commits، Pull Request و چک‌لیست بازبینی ML

tag و SemVer به‌عنوان «Model Release»؛ Gitea در برابر GitHub

لایه‌های کیفیت: ruff (lint/format)، pytest (چه چیزی را در کد ML تست کنیم)، pre-commit آفلاین

بلاک ۲ — Lab (۷۵ دقیقه)

Gitea بالا، ساخت کاربر و Repo، اولین push (۱۵ دقیقه)

افزودن و تنظیم ruff و pytest (۱۳ دقیقه)

شاخه‌ی feature جدید، تست‌های starter و نوشتن تست چهارم (۱۵ دقیقه)

نصب hook و آزمایش مسدود شدن فایل env و کلید hard-code (۱۰ دقیقه)

PR، بازبینی، merge و tag نسخه‌ی `v0.1.0` (۲۰ دقیقه)

خروجی / Definition of Done

خروجی: Repo روی Gitea با ۴ آزمون، hook فعال و tag

Repo روی Gitea است و main در آن push شده

ruff check و ruff format بدون خطا

pytest چهار تست پاس‌شده نشان می‌دهد

hookها نصب‌اند و فایل محرمانه و کلید hard-code را مسدود می‌کنند

PR مرج شده و tag `v0.1.0` روی Gitea دیده می‌شود

تکلیف

یک آزمون دیگر بنویسید (با commit از نوع test)

بخش «Contributing» را در README اضافه کند (نام‌گذاری branch، Conventional Commits، چک‌لیست PR)

هدف‌های lint و test را به Makefile اضافه کند

Gitea و Docker را روشن نگه دارد

ابزارها: Git، Gitea، ruff، pytest، pre-commit

پیش‌نیاز از جلسه‌ی قبل: Repo و baseline جلسه‌ی ۱؛ Gitea از lab kit بالا آمده است

جلسه ۳ — مبانی Docker و ذخیره‌ساز شیء On-Prem

Session 3 · Docker Essentials + Object Storage (SeaweedFS)

اهداف جلسه

در پایان این جلسه می‌توانید:

یک کانتینر را اجرا، بررسی و حذف کند و بداند چه چیزی با حذف آن از بین می‌رود

فایل Compose را بخواند و با آن سرویس‌ها را بالا بیاورد؛ تفاوت `down` و `down -v` را بداند

تفاوت ذخیره‌ساز فایل، بلوک و شیء و اجزای آدرس S3 را توضیح دهد

با AWS CLI به SeaweedFS وصل شود، bucket بسازد و فایل را با checksum بسنجد

بلاک ۱ — مفاهیم و Demo (۶۵ دقیقه)

چرا کانتینر (works on my machine)؛ VM در برابر container؛ image، container، registry، tag

چرخه‌ی عمر کانتینر، پورت و شبکه، volume در برابر bind mount، خواندن Dockerfile، Docker Compose

واقعیت آفلاین: `docker load` از بسته‌ی دوره

ذخیره‌ساز فایل/بلوک/شیء؛ واژگان S3 (bucket، key، endpoint، access key)؛ چرا S3-compatible برای MLOps مهم است

معرفی SeaweedFS (در حد استفاده) و AWS CLI روی endpoint محلی

بلاک ۲ — Lab (۷۵ دقیقه)

preflight و اولین کانتینرها (چرخه‌ی run تا rm) (۱۲ دقیقه)

آزمایش پورت و volume: داده بدون volume می‌رود، با volume می‌ماند (۱۳ دقیقه)

راه‌اندازی SeaweedFS با Docker Compose (۱۲ دقیقه)

S3 با AWS CLI: دو bucket (dvc و mlflow)، آپلود و دانلود، مقایسه‌ی checksum (۱۶ دقیقه)

ماندگاری (down و up)، `env.example` در Git و env واقعی بیرون از Git، PR (۲۰ دقیقه)

خروجی / Definition of Done

خروجی: S3 endpoint داخلی آماده + دو bucket

SeaweedFS healthy است و پورت ۸۳۳۳ جواب می‌دهد

bucketهای dvc و mlflow وجود دارند

`churn.csv` رفته و برگشته و checksum دو فایل یکسان است

بعد از compose down و up داده هنوز هست

`env.example` در Git است، فایل env نیست، و PR مرج شده

تکلیف

بخش «Local storage» در README (بالا آوردن SeaweedFS، ساخت env از نمونه، ساخت دو bucket)

هدف `s3-ls` در Makefile، با PR

جدول خطاهای عمدی (کلید و پورت اشتباه) و پیام‌ها در README

ابزارها: Docker، Docker Compose، SeaweedFS (S3)، AWS CLI

پیش‌نیاز از جلسه‌ی قبل: Repo روی Gitea با آزمون‌ها و hook (جلسه‌ی ۲)؛ Docker روشن

جلسه ۴ — نسخه‌بندی داده با DVC

Session 4 · Data Versioning with DVC

اهداف جلسه

در پایان این جلسه می‌توانید:

جایگاه داده در MLOps (منبع، lineage، خطر نبودن نسخه‌گذاری) را توضیح دهد

داده را با `dvc add` ردیابی کند و فایل pointer را بخواند؛ بداند بایت‌ها کجا می‌روند

remote روی SeaweedFS بسازد، رمز را بیرون از Git نگه دارد و داده را push و pull کند

با `git checkout` + `dvc checkout` به نسخه‌ی ۱ برگردد و داده‌ی حذف‌شده را بازیابی کند

بلاک ۱ — مفاهیم و Demo (۶۵ دقیقه)

زمینه‌ی مهندسی داده: batch در برابر stream، lake/warehouse، lineage

چرا Git برای داده کافی نیست؛ ایده‌ی DVC: pointer در Git، بایت‌ها در cache و remote

remote روی SeaweedFS و نگه‌داشتن رمز بیرون از Git (config.local)

push/pull/status، زمان‌سفر دوگامی، `dvc diff` و tag نسخه‌ی داده

بلاک ۲ — Lab (۷۵ دقیقه)

بررسی SeaweedFS و bucket، نصب و `dvc init` روی شاخه (۱۰ دقیقه)

ردیابی dataset با `dvc add`، خواندن pointer و cache، commit (۱۲ دقیقه)

اتصال remote با رمز محلی، push و دیدن شیء در bucket (۱۳ دقیقه)

ساخت نسخه‌ی ۲ داده (۸۰۰۰ ردیف با drift)، push، diff، مقایسه‌ی متریک نسخه‌ها (۱۶ دقیقه)

بازگشت به نسخه‌ی ۱، بازیابی پس از حذف با `dvc pull`، PR و tag (۱۷ دقیقه)

خروجی / Definition of Done

خروجی: DVC remote فعال + دو نسخه‌ی داده با tag (`data-v1`، `data-v2`)

pointer در Git است و فایل داده از Git بیرون است

remote به نام storage روی bucket به نام dvc تنظیم شده و رمز فقط محلی است

دو نسخه‌ی داده با tagهای `data-v1` و `data-v2`، هر دو در remote

بازگشت به نسخه‌ی ۱: checksum و متریک اول دوباره دیده شد

بازیابی بعد از حذف داده و cache با pull، و PR مرج شده

تکلیف

نسخه‌ی ۳ داده با seed دیگر، add، push، tag، و جدول متریک سه نسخه در README

بخش «Data versioning» در README (clone همکار تازه، رمز محلی، pull)

هدف‌های `data-pull` و `data-status` در Makefile، با PR

شکست عمدی: پورت remote یا مقدار local را خراب کند و پیام‌ها را در جدول README بنویسد

ابزارها: DVC، SeaweedFS (S3)، Git

پیش‌نیاز از جلسه‌ی قبل: SeaweedFS healthy و دو bucket dvc و mlflow (جلسه‌ی ۳)

جلسه ۵ — DVC Pipeline و بازتولیدپذیری

Session 5 · DVC Pipeline (preprocess/train/evaluate) & Reproducibility

اهداف جلسه

در پایان این جلسه می‌توانید:

فایل `dvc.yaml` را با stage، deps، outs، params و metrics طراحی و گراف وابستگی (DAG) را بخواند

با `dvc repro` فقط مراحل تغییرکرده را اجرا کند

با `dvc exp` و diff، اثر تغییر params را بر metrics مقایسه کند

تفاوت مفهومی DVC با Airflow، Kubeflow و Argo را توضیح دهد

بلاک ۱ — مفاهیم و Demo (۶۵ دقیقه)

طراحی `dvc.yaml`: stage، deps، outs، params، metrics و گراف وابستگی (DAG)

`dvc repro` و اجرای هوشمند؛ `dvc exp` و مقایسه‌ی params/metrics

مفهوم orchestration: DVC در برابر Airflow، Kubeflow، Argo (فقط مقایسه‌ی مفهومی)

بلاک ۲ — Lab (۷۵ دقیقه)

ساخت pipeline کامل `preprocess → train → evaluate`

اجرای `dvc repro`، تغییر پارامتر و مشاهده‌ی params diff و metrics diff

خروجی / Definition of Done

خروجی: pipeline تکرارپذیر با خروجی متریک

`dvc repro` pipeline را از ابتدا تا انتها اجرا می‌کند و خروجی متریک می‌دهد

خروجی verify جلسه و commit hash ارسال شده است

تکلیف

یک پارامتر اضافه کند و اثر آن را ثبت کند

ابزارها: DVC، SeaweedFS (S3)، uv

پیش‌نیاز از جلسه‌ی قبل: DVC remote فعال و دو نسخه‌ی داده با tag (جلسه‌ی ۴)

جلسه ۶ — MLflow Tracking و اتصال به پروژه

Session 6 · Experiment Tracking with MLflow

اهداف جلسه

در پایان این جلسه می‌توانید:

مفاهیم experiment، run، پارامتر، متریک و artifact را توضیح دهد

معماری tracking server (backend store و artifact store) را شرح دهد

runهای pipeline را در MLflow ثبت و در UI مقایسه کند

بلاک ۱ — مفاهیم و Demo (۶۵ دقیقه)

Experiment Tracking: experiment، run، پارامتر، متریک، artifact

معماری tracking server: backend store (SQLite) و artifact store (SeaweedFS از روز اول)

بلاک ۲ — Lab (۷۵ دقیقه)

بالا آوردن MLflow server با Compose

لاگ کردن runها از pipeline و مقایسه‌ی آن‌ها در UI

خروجی / Definition of Done

خروجی: MLflow UI + چند run ثبت‌شده با artifact روی S3

MLflow server بالا است و runها با artifact روی S3 در UI دیده می‌شوند

خروجی verify جلسه و commit hash ارسال شده است

تکلیف

artifactهای کلیدی (مدل و `metrics.json`) را ثبت و سه run را مقایسه کند

ابزارها: MLflow، Docker Compose، SQLite، SeaweedFS (bucket mlflow)، DVC

پیش‌نیاز از جلسه‌ی قبل: pipeline تکرارپذیر (جلسه‌ی ۵) و bucket mlflow (جلسه‌ی ۳)

جلسه ۷ — Model Registry و Promotion

Session 7 · Model Registry & Promotion

اهداف جلسه

در پایان این جلسه می‌توانید:

- Registered model، version، alias (`@champion`، `@challenger`) و tag را به‌کار ببرد و بداند Stageها روش قدیمی‌اند
- signature مدل و release note / model card را توضیح دهد
- ارزیابی را به‌عنوان دروازه‌ی انتشار (evaluation gate) پیاده کند

بلاک ۱ — مفاهیم و Demo (۶۵ دقیقه)

- Registered model، version، alias (`@champion` / `@challenger`) و tag؛ Stageها فقط به‌عنوان روش قدیمی (منسوخ از MLflow 2.9)
- Model signature، release note / model card
- ارزیابی به‌عنوان دروازه‌ی انتشار (evaluation gate)

بلاک ۲ — Lab (۷۵ دقیقه)

- ثبت مدل در Registry
- اسکریپت promotion: `@challenger → @champion` با شرط آستانه‌ی متریک
- اختیاری: نمودار SHAP به‌عنوان artifact

خروجی / Definition of Done

خروجی: مدل ثبت‌شده با alias + دروازه‌ی ارزیابی

- مدل در Registry با alias `@champion` ثبت شده و promotion فقط با عبور از آستانه‌ی متریک انجام می‌شود
- خروجی verify جلسه و commit hash ارسال شده است

تکلیف

- Release Note کوتاه برای مدل بنویسد (چه چیزی تغییر کرد)

ابزارها: MLflow (Registry)، SHAP (اختیاری)

پیش‌نیاز از جلسه‌ی قبل: MLflow server و runهای ثبت‌شده (جلسه‌ی ۶)

جلسه ۸ — سرویس Inference با FastAPI

Session 8 · Model Serving with FastAPI

اهداف جلسه

در پایان این جلسه می‌توانید:

سرویس REST با schema ورودی pydantic و endpointهای health/readiness بسازید

logging ساخت‌یافته و endpoint متریک `/metrics` را اضافه کنید

قرارداد ورودی را با signature مدل هم‌خوان کنید

مدل را با alias از Registry بار کنید و سرویس را با pytest بیازمایید

بلاک ۱ — مفاهیم و Demo (۶۵ دقیقه)

Serving basics: REST، schema ورودی با pydantic، health/readiness

logging ساخت‌یافته و endpoint متریک (`/metrics`)

برای جلسه‌ی ۱۳

قرارداد ورودی: هم‌خوانی pydantic با signature مدل

بلاک ۲ — Lab (۷۵ دقیقه)

پیاده‌سازی API که مدل را با alias از Registry بار می‌کند (`models:/churn@champion`)

آزمون با pytest و httpx

خروجی / Definition of Done

خروجی: endpoint فعال و آزمون‌شده

API مدل `@champion` را بار می‌کند و پاسخ می‌دهد

آزمون‌های pytest و httpx پاس می‌شوند

خروجی verify جلسه و commit hash ارسال شده است

تکلیف

الگوی batch inference (اختیاری)

ابزارها: FastAPI، pydantic، httpx، pytest، MLflow

پیش‌نیاز از جلسه‌ی قبل: مدل ثبت‌شده با alias `@champion` (جلسه‌ی ۷)

جلسه ۹ — Containerization و امنیت image

Session 9 · Containerization & Image Security

اهداف جلسه

در پایان این جلسه می‌توانید:

Dockerfile را با بهترین رویه‌ها (multi-stage، base سبک، کاربر non-root، dockerignore) بنویسید

image را با Trivy (با پایگاه‌داده‌ی آفلاین) اسکن کند و یافته‌ها را اصلاح کند

SBOM و نقش registry محلی را در زنجیره‌ی تأمین توضیح دهد

بلاک ۱ — مفاهیم و Demo (۶۵ دقیقه)

Dockerfile best practices: multi-stage، base سبک، کاربر non-root، dockerignore

Trivy scan (با پایگاه‌داده‌ی آفلاین)، SBOM، registry محلی

بلاک ۲ — Lab (۷۵ دقیقه)

Dockerize کردن API و آزمون محلی

اسکن با Trivy و اصلاح حداقل ۲ مورد

push به registry محلی

خروجی / Definition of Done

خروجی: image امن‌تر و نسخه‌دار در registry محلی

image با کاربر non-root ساخته شده و حداقل ۲ یافته‌ی Trivy اصلاح شده است

image نسخه‌دار در registry محلی push شده است

خروجی verify جلسه و commit hash ارسال شده است

تکلیف

تولید SBOM با Syft (اختیاری) و مستند کردن یافته‌های باقی‌مانده

ابزارها: Docker، Trivy، Syft (اختیاری)، registry محلی

پیش‌نیاز از جلسه‌ی قبل: API فعال و آزمون‌شده (جلسه‌ی ۸)

جلسه ۱۰ — Deploy روی VM و Infrastructure as Code

Session 10 · VM Deployment + Infrastructure as Code

اهداف جلسه

در پایان این جلسه می‌توانید:

توضیح دهد چرا VM هنوز در محیط‌های سازمانی و legacy مهم است

سرویس را با systemd و reverse proxy (Nginx) و log rotation بیاراید

اصول IaC و idempotency را با Ansible (و مرور Terraform) به‌کار ببرد

rollback را با اجرای دوباره‌ی playbook با نسخه‌ی قبلی انجام دهد

بلاک ۱ — مفاهیم و Demo (۶۵ دقیقه)

چرا هنوز VM مهم است (legacy/سازمانی)

log + reverse proxy (Nginx) + systemd service rotation

اصول IaC: Ansible (و مرور Terraform)، idempotency

بلاک ۲ — Lab (۷۵ دقیقه)

playbook Ansible که سرویس مدل را روی VM (Multipass) مستقر می‌کند

نوشتن Runbook حداقلی: start، stop، restart، check logs

تمرین rollback: اجرای دوباره‌ی playbook با نسخه‌ی قبلی

خروجی / Definition of Done

خروجی: سرویس روی VM + playbook + Runbook

سرویس با playbook روی VM مستقر و در دسترس است

Runbook حداقلی نوشته شده و rollback با نسخه‌ی قبلی انجام شده است

خروجی verify جلسه و commit hash ارسال شده است

تکلیف

سناریوی rollback را مستند کند

ابزارها: Ansible، Multipass، systemd، Nginx

پیش‌نیاز از جلسه‌ی قبل: سرویس مدل و نسخه‌ی بسته‌بندی‌شده‌ی آن (جلسه‌های ۸ و ۹)

جلسه ۱۱ — Kubernetes Deploy

Session 11 · Kubernetes Deploy (Deployment/Service/Config/Probes/Resources)

اهداف جلسه

در پایان این جلسه می‌توانید:

اجزای ضروری K8s برای سرویس مدل (pod، deployment، service، configmap، secret) را بشناسد

readiness/liveness، requests/limits و rollout را تنظیم کند

سرویس را روی kind مستقر و با port-forward، logs و events بررسی کند

بلاک ۱ — مفاهیم و Demo (۶۵ دقیقه)

اجزای ضروری K8s برای سرویس مدل: pod، deployment، service، configmap، secret

readiness/liveness، requests/limits، rollout

بلاک ۲ — Lab (۷۵ دقیقه)

deploy سرویس مدل روی kind

تست با port-forward و بررسی logs/events

تمرین `rollout undo`

خروجی / Definition of Done

خروجی: سرویس running روی K8s

سرویس روی kind در وضعیت running است و با port-forward جواب می‌دهد

`rollout undo` تمرین شده است

خروجی verify جلسه و commit hash ارسال شده است

تکلیف

resource limits و probes را اضافه و نتیجه را ثبت کند

ابزارها: Kubernetes (kind)، kubectl، registry محلی

پیش‌نیاز از جلسه‌ی قبل: image نسخه‌دار در registry محلی (جلسه‌ی ۹)

جلسه ۱۲ — Ingress و Helm

Session 12 · Ingress & Helm

اهداف جلسه

در پایان این جلسه می‌توانید:

سرویس را با Ingress روی Traefik expose کند و Gateway API را به‌عنوان مسیر آینده بشناسد

ساختار chart و values هر محیط را توضیح دهد

با Helm بسته‌بندی، upgrade و rollback انجام دهد

بلاک ۱ — مفاهیم و Demo (۶۵ دقیقه)

Ingress با Traefik و expose کردن سرویس؛ پیش‌نمای Gateway API

Helm basics: ساختار chart، values برای هر محیط

بلاک ۲ — Lab (۷۵ دقیقه)

اضافه کردن Ingress با مسیریابی بر اساس host

بسته‌بندی با Helm، `values-dev` و `values-prod`

rollback و upgrade

خروجی / Definition of Done

خروجی: سرویس با Helm chart + Ingress

سرویس از طریق Ingress (مسیر host) در دسترس است

chart با `values-dev` و `values-prod` نصب، upgrade و rollback شده است

خروجی verify جلسه و commit hash ارسال شده است

تکلیف

یک مقدار قابل‌تنظیم به chart اضافه و مستند کند

ابزارها: Traefik، Helm، kind

پیش‌نیاز از جلسه‌ی قبل: سرویس running روی kind (جلسه‌ی ۱۱)

جلسه ۱۳ — CI آگاه از ML، Monitoring و Drift

Session 13 · ML-aware CI + Monitoring + Drift

اهداف جلسه

در پایان این جلسه می‌توانید:

CI پایه (lint، test، build، scan، push) را روی Gitea Actions بسازد

CI مخصوص ML را اضافه کند: گزارش metrics-diff روی PR و retrain زمان‌بندی‌شده

با متریک‌های RED و Prometheus/Grafana سرویس را پایش کند

data drift را بشناسد و با Evidently گزارش بسازد

بلاک ۱ — مفاهیم و Demo (۶۵ دقیقه)

CI پایه: `lint → test → build → scan → push` روی Gitea Actions

CI مخصوص ML: گزارش metrics-diff روی PR، retrain زمان‌بندی‌شده

Monitoring: متریک‌های RED، Prometheus/Grafana

Data drift و معرفی Evidently

بلاک ۲ — Lab (۷۵ دقیقه)

ساخت pipeline CI + نظر خودکار metrics-diff روی PR + workflow زمان‌بندی‌شده‌ی retrain

داشبورد Grafana برای latency، error و RPS

گزارش drift با Evidently (داده‌ی نسخه‌ی ۱ در برابر ۲)

خروجی / Definition of Done

خروجی: CI عملیاتی + drift report + dashboard

CI روی Gitea Actions اجرا می‌شود و metrics-diff روی PR نظر می‌گذارد

داشبورد Grafana (latency، error، RPS) و drift report ساخته شده‌اند

خروجی verify جلسه و commit hash ارسال شده است

تکلیف

یک alert ساده تعریف کند و Demo را آماده کند

ابزارها: Gitea Actions، Prometheus، Grafana، Evidently، DVC

پیش‌نیاز از جلسه‌ی قبل: Helm chart و سرویس روی K8s (جلسه‌ی ۱۲)، `/metrics` (جلسه‌ی ۸)، دو نسخه‌ی داده (جلسه‌ی ۴)

جلسه ۱۴ — جمع‌بندی و Demo پروژه‌ی پایانی (Mini Production)

Session 14 · Capstone Integration & Demo

اهداف جلسه

در پایان این جلسه می‌توانید:

پروژه را با چک‌لیست امتیازدار Production readiness ارزیابی کند

پروژه را سرتاسر از clone تمیز اجرا کند (DVC، MLflow، Docker، K8s، Monitoring)

Demo و دفاع کوتاه ارائه دهد و پروژه را برای رزومه روایت کند

نقشه‌ی راه و نگاشت On-Prem به Cloud را مرور و مسیر ادامه را بشناسد

بلاک ۱ — مفاهیم و Demo (۶۵ دقیقه)

چک‌لیست Production readiness (امتیازدار)

آماده‌سازی Demo و Storytelling پروژه برای رزومه

مرور نقشه‌ی راه (roadmap.sh/mlops) و نگاشت On-Prem به Cloud؛ پیش‌نمای دوره‌ی ۲

بلاک ۲ — Demo / Lab (۷۵ دقیقه)

اجرای سرتاسری از clone تمیز (DVC، MLflow، Docker، K8s، Monitoring)

Demo و دفاع کوتاه + رفع اشکال

خودارزیابی نهایی با roadmap.sh/mlops

خروجی / Definition of Done

خروجی: Repo نهایی + Demo + Runbook

Repo نهایی از clone تمیز سرتاسر اجرا می‌شود

Runbook نهایی و Demo تحویل شده است

خروجی verify جلسه و commit hash ارسال شده است

تکلیف

اختیاری: ضبط ویدئوی ۳ دقیقه‌ای Demo برای رزومه

ابزارها: همه‌ی ابزارهای دوره

پیش‌نیاز از جلسه‌ی قبل: همه‌ی خروجی‌های جلسه‌های ۱ تا ۱۳ در Repo

۸ ارزیابی
Assessment

ارزیابی کاملاً عملی است و بر شواهد قابل‌بررسی در Repo تکیه دارد.

وزن	مؤلفه	شاهد	زمان
۲۰٪	**Checkpointهای هر جلسه**	خروجی اسکریپت verify جلسه + commit hash	پایان هر جلسه
۳۰٪	**تمرین‌های Lab**	چک‌لیست Definition of Done هر جلسه	پایان هر جلسه
۵۰٪	**پروژه‌ی پایانی**	Repo نهایی، Runbook و Demo (جدول زیر)	جلسه‌ی ۱۴

چطور checkpointها مستند می‌شوند

- Checkpoint جلسه: دانشجو خروجی اسکریپت verify (مثلاً `verify.sh s01`) و commit hash را برای مدرس می‌فرستد؛ مدرس با همان hash در Repo یا از tag مرجع بررسی می‌کند.
- Lab: هر جلسه یک چک‌لیست Definition of Done دارد که دانشجو در پایان جلسه با مدرس بررسی می‌کند.
- پروژه‌ی پایانی: مدرس در جلسه‌ی ۱۴ Repo را از clone تمیز اجرا می‌کند و بر اساس چک‌لیست Production readiness امتیاز می‌دهد.

جدول امتیاز پروژه‌ی پایانی (۵۰٪ نمره)

معیار	امتیاز	ملاک
اجرا از clone تمیز	۱۵	Repo از clone تمیز و بدون دستکاری دستی سرتاسر اجرا می‌شود
pipeline تکرارپذیر	۱۰	`dvc repro` با داده و params نسخه‌دار، خروجی متریک یکسان
Registry و promotion با alias	۵	مدل ثبت‌شده، alias `@champion` با دروازه‌ی ارزیابی
image امن	۵	multi-stage، non-root، اسکن Trivy، یافته‌های اصلاح یا مستند شده، image در registry محلی
Ingress + Helm + K8s	۵	استقرار روی kind با Helm chart و (Traefik) Ingress، probe و resources
drift report + Monitoring	۵	داشبورد Grafana (latency، error، RPS) و گزارش drift با Evidently
Demo + Runbook	۵	Runbook اجرای پروژه، Demo و دفاع کوتاه
جمع	۵۰	

۹ خروجی نهایی دوره

Final Deliverable

در پایان دوره، Repo پروژه‌ی Churn Mini Production شامل این‌هاست:

- کد آزمون‌شده با ruff، pytest و pre-commit و تاریخچه‌ی PR و tag روی Gitea
- DVC remote روی SeaweedFS S3 و pipeline تکرارپذیر (`dvc.yaml`)
- MLflow tracking و registry با alias
- سرویس inference (FastAPI) در Docker، اسکن Trivy و registry محلی
- Deploy روی VM با Ansible و روی Kubernetes با Helm و Ingress (Traefik)
- CI روی Gitea Actions، Monitoring پایه (Dashboard) و Drift report
- Runbook اجرای پروژه

۱۰ هم‌ترازی با roadmap.sh/mlops

Alignment with roadmap.sh/mlops

سطح هدف: ۰ = پوشش داده نمی‌شود، ۱ = آشنایی (توضیح و Demo)، ۲ = هدایت‌شده (Lab عملی با دستورالعمل)، ۳ = مستقل (بدون workbook، شاهد در پروژه‌ی پایانی).

موضوع roadmap	جلسه	سطح هدف
What is MLOps؟ (اصول و اجزا)	۱	۳
Programming Basics (Bash؛ Python پیش‌نیاز است)	۰	۲
Version Control (Git، Gitea، DVC)	۲، ۴، ۵	۳
CI/CD (Gitea Actions، metrics-diff روی PR)	۱۳	۲ تا ۳
ML Fundamentals (ارزیابی به‌عنوان دروازه، MLflow)	۶، ۷	۲ تا ۳
Cloud Computing (فقط نگاشت On-Prem به Cloud)	۱۴	۱
Containerization (Docker؛ Kubernetes)	۳، ۹؛ ۱۱، ۱۲	۲؛۳
Data Engineering Basics (batch/stream، lake/warehouse، lineage)	۴	۱
Infrastructure as Code (Ansible؛ مرور Terraform)	۱۰	۲
Orchestration (مفهوم DAG؛ retrain زمان‌بندی‌شده)	۵، ۱۳	۱ تا ۲
Observability & Monitoring (Prometheus، Grafana، drift)	۱۳	۲
Explainable AI (SHAP)، تکلیف اختیاری	۷	۱
Edge AI	—	۰ (خارج از دامنه)

نتیجه: دوره در هر ۷ جزء اصلی MLOps (Version Control، CI/CD، Orchestration، Experiment Tracking، Data Lineage، Model Serving & Training، Monitoring) به سطح ۲ تا ۳ می‌رسد و در ۱۰ مورد از ۱۳ موضوع roadmap دست‌کم سطح ۱ دارد. Edge AI عمداً بیرون از دامنه است.

۱۱ وضعیت تولید محتوا

Content Production Status

وضعیت در تاریخ ۱۳-۱۰-۲۰۲۶:

جلسه‌های ۱ تا ۴	اسلاید و یادداشت مدرس آماده؛ آزمایشگاه‌ها اجرا شده‌اند
کتابچه‌ی واژه‌نامه	آماده
جلسه‌های ۵ تا ۱۴ و جلسه‌ی ۰	هنوز ساخته نشده‌اند
lab kit و نسخه‌ها	نسخه‌ها قفل شده‌اند (۰۸-۱۰-۲۰۲۶)؛ ساخت کامل بسته‌ی آفلاین هنوز اجرا نشده و برای ۱۴ روز پیش از شروع دوره برنامه‌ریزی شده است

جزئیات آزمایشگاه جلسه‌های ۵ تا ۱۴ و جلسه‌ی ۰ در این سند طبق نقشه‌ی دوره نوشته شده است و ممکن است هنگام ساخت محتوا تغییر کند. زمان‌بندی گام‌های Lab این جلسه‌ها هنگام ساخت تعیین می‌شود.

مسیر ادامه: دوره‌ی ۲

- دوره‌ی ۲: Enterprise On-Prem & MLOps Production، ۴۵ ساعت (۱۳ جلسه‌ی ۳٫۵ ساعته + یک جلسه‌ی کلینیک/دفاع).
- Compute، Serving و Monitoring روی Kubernetes On-Prem؛ Storage و artifactها روی AWS S3 واقعی.
- موضوع‌ها: RBAC و سخت‌سازی K8s، Feature Store با Feast، CI/CD و GitOps با ArgoCD، KServe با Canary، Drift پیشرفته، HITL و Governance (OPA) و نگاشت به SageMaker.